

Level 1 Caching in Hibernate

Level 1 (L1) Cache is the default cache mechanism provided by Hibernate for each session. It caches the objects during a session's lifespan, ensuring that multiple requests for the same object within a session do not result in multiple database queries. This cache is **session-scoped**, meaning it is only valid for the lifespan of a single session.

Key Points of Level 1 Cache:

1. **Session-Scope:** Each Hibernate session has its own Level 1 cache. Objects retrieved during a session are stored in that cache, and any subsequent requests for the same object will be served from the cache, avoiding unnecessary database hits.
2. **Caching Objects:** Objects are stored in the cache once they are retrieved using methods like `get()`, `load()`, or `getReference()`. If an object is already in the cache, Hibernate will return it directly without querying the database again.
3. **Automatic Cache Management:** Hibernate automatically manages the cache for the session. The user does not have to explicitly put or remove objects from the cache.
4. **Flush and Clear:** The cache gets cleared when the session is closed, or when the session is flushed or cleared manually. `session.flush()` writes changes to the database, and `session.clear()` removes all objects from the cache.

Flow of Data in Level 1 Cache:

- When a query is made for an object using `get()` or `load()`, Hibernate checks if the object is already present in the session's cache:
 - **If Present:** It returns the object from the cache (without hitting the database).
 - **If Absent:** It fetches the object from the database and stores it in the cache for future use.

Example: Level 1 Caching with Student EntityCode:

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Transient;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id
    @Column(name="sid")
    private int id;

    @Column(name="sname")
    private String name;

    @Transient
    private String city;

    public Student() {
        System.out.println("Zero param constructor");
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }
}
```

```

    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", city=" + city +
    "]\n";
    }
}

```

Main.java

```

import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {

```

```

public static void main(String[] args) {
    Configuration config = null;
    SessionFactory sessionFactory = null;
    Session session1 = null;
    Session session2=null;
    Transaction transaction = null;
    boolean flag = false;
    try {
        config = new Configuration();
        config.configure();
        sessionFactory = config.buildSessionFactory();
        session1 = sessionFactory.openSession();

        Student s1=session1.get(Student.class, 12);
        Student s2=session1.get(Student.class, 12);
        System.out.println(s1);
        System.out.println(s2);

        session2=sessionFactory.openSession();
        Student s3=session2.get(Student.class, 12);
        Student s4=session2.get(Student.class, 12);
        System.out.println(s3);
        System.out.println(s4);

    } catch (HibernateException e) {
        System.out.println("Hibernate exception: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected exception: " + e.getMessage());
    } finally {

        if (session1 != null) {
            session1.close();
        }
        if (session2 != null) {

```

```

        session2.close();
    }
    if (sessionFactory != null) {
        sessionFactory.close();
    }
}
}
}

```

Output:

```

Hibernate:
    select
        s1_0.sid,
        s1_0.sname
    from
        StudentTable s1_0
    where
        s1_0.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]
Hibernate:
    select
        s1_0.sid,
        s1_0.sname
    from
        StudentTable s1_0
    where
        s1_0.sid=?
Zero param constructor
Student [id=12, name=Shiva, city=null]
Student [id=12, name=Shiva, city=null]

```

Notes:

1. For each session, only one call is made to the database for the same object, regardless of how many times the object is accessed.
2. When the session retrieves an object for the first time with a particular ID, the object is instantiated using the zero-argument constructor, and its values are populated using setter methods.

Explanation:

1. Session 1:

- The first time `session1.get(Student.class, 1)` is called, Hibernate queries the database, retrieves the student object, and stores it in the **L1 cache** for session1.
- When `session1.get(Student.class, 1)` is called again, Hibernate does **not hit the database**. It fetches the object from the cache and returns it.

2. Session 2:

- A new session (session2) is opened, which means a new **L1 cache** is created for session2.
- When `session2.get(Student.class, 1)` is called, Hibernate queries the database again because the cache is specific to session1. Objects in session1's cache are not available in session2.

Important Points to Remember:

1. **Each Session has its own Cache:** The cache is **session-specific**, so it does not persist across multiple sessions. Each session will have its own L1 cache.
2. **No Need for Configuration:** L1 cache is enabled by default, and no configuration is required.
3. **Cache Lifetime:** The L1 cache exists for the duration of a session. Once the session is closed, the cache is cleared.
4. **Flush and Clear:** You can clear the cache manually using `session.clear()`, or flush the session using `session.flush()`.

Object Storage in Level 1 Cache:

- The **L1 cache** stores objects as **entity objects** mapped with their unique primary keys.
- The session checks the cache before querying the database, reducing unnecessary database interactions and improving performance.

Level 1 Cache Flow:

1. **Query Object:** The session.get() or session.load() is called.
2. **Check Cache:** Hibernate checks if the requested object is already present in the L1 cache of the session.
3. **Cache Hit:** If found, the object is returned from the cache.
4. **Cache Miss:** If not found, the database query is executed, the object is fetched and stored in the cache.

Output from Example:

1. **First session (session1):**
 - First get() call fetches the object from the database and stores it in the session's cache.
 - Second get() call retrieves the object from the cache (no database hit).
 2. **Second session (session2):**
 - Since it's a new session, the cache is empty. A new database query is made to fetch the object.
- Notice how the database query is executed only once in **session1**. The subsequent call fetches the object from the cache. However, in **session2**, a new database query is made, showing that the L1 cache is session-specific.